



Phone | [Email](#) | [Linkedin](#) | [Github](#) | [Portfolio](#)

PROFESSIONAL SUMMARY

Software engineer specializing in scalable backend systems and AI applications. Worked on production microservices for video streaming and location chatbot solutions. Excel in fast-paced, high-impact technical environments.

EDUCATION

Uni 1 <i>M.S. Computer Science – GPA: 3.85/4.0</i>	Location 1 <i>Aug. 2024 – May 2026</i>
Uni 2 <i>B.S Computer Engineering</i>	Location 2 <i>Jul. 2018 – Jul. 2022</i>

TECHNICAL SKILLS

Languages: Python, SQL, Bash

Backend Engineering: FastAPI, RESTful APIs, JWT, OAuth2, GraphQL, Microservices Architecture

Cloud & DevOps: AWS (Lambda, EC2, S3, SQS, ECR), boto3 AWS SDK, Docker, CI/CD

AI & Machine Learning: RAG (Retrieval-Augmented Generation), Large Language Models(LLMs), LangChain, LangGraph, Vector Embeddings, PyTorch

Databases: PostgreSQL, SQLite, MongoDB

PROFESSIONAL EXPERIENCE

Software Intern <i>Company 1</i>	Jul. 2025 – Aug. 2025 <i>Location 1</i>
- Designed the backend for a location-aware Large Language Model(LLM) chatbot, supporting natural language queries including proximity search, rating-based ranking, category filtering. - Reduced hallucinated responses by grounding outputs in OpenStreetMap data using a Retrieval-Augmented Generation (RAG) system. - Achieved sub 200 ms radius-based spatial queries by ingesting 2000+ OpenStreetMap POIs into PostgreSQL using PostGIS extension. - Enabled conversational state and multi-turn query refinement across sessions by building multi-step LLM workflows using LangChain and LangGraph.	
Software Developer <i>Company 2</i>	Aug. 2022 – Sept. 2023 <i>Location 2</i>
- Built the backend for a multi-client video streaming content management system(CMS). - Reduced new client onboarding effort by designing a configuration-driven architecture using JSON-based client settings. - Delivered 15+ RESTful low-latency APIs, GraphQL queries and well-documented endpoints using FastAPI. - Secured 100% of CMS and administrative endpoints and enabled role-based access control across tenant environments by implementing JWT-based authentication with OAuth2. - Managed data storage using PostgreSQL and MongoDB, modeling and querying 10+ relational entities while supporting flexible, schema-less client configurations. - Improved local setup time by 50% and ensured consistent environments across development and staging by containerizing backend services using Docker.	

PROJECTS

Elastic Face Recognition (Serverful/IaaS) | AWS (EC2, SQS, S3), boto3 AWS SDK, Python, PyTorch

- Built a production-ready distributed ML inference system using PyTorch deep learning models deployed across auto-scaling EC2 instances with S3 persistence for input images and classification results.
- Enabled on-demand scalability from 0 to 15 concurrent instances by designing a custom autoscaling controller that dynamically provisions EC2 app-tier instances based on SQS queue depth, eliminating idle resource costs.
- Achieved real-time face recognition inference as measured by per-request classification responses by using PyTorch deep learning models deployed on custom EC2 AMIs with CPU-optimized torch installations.

Serverless Face Recognition (Lambda/FaaS) | AWS (Lambda, SQS, ECR, boto3 AWS SDK, Python, PyTorch, Docker

- Processed video frame streams in real-time by implementing a two-stage pipeline: MTCNN-based face detection triggering ResNet/VGGFace2 recognition via SQS event-driven architecture.
- Reduced infrastructure management overhead to zero by architecting a fully serverless face recognition pipeline using AWS Lambda, SQS, and ECR with automatic scaling and pay-per-invocation pricing.
- Enabled seamless client integration by exposing face detection via Lambda Function URLs accepting base64-encoded frames and returning recognition results through SQS response queues.